

COMPUTER VISION - FIB UPC

Phase 1 · Short Project Definition

Visual Ingredient Scanner with Recipe Generation

An on-device Computer Vision pipeline that detects food items, estimates their quantity using monocular depth, and suggests recipes.

Project type	Productivity (build tools that work)
Course	Computer Vision · Master MEI FIB · UPC · Spring 2026
Team	Pol Plana Emma Nájera Houda El Fezzak
Delivery	Phase 1: Short Project Definition

1. Problem statement & motivation

Even though meal planning seems trivial, a lot of households struggle to optimally use up the food they have stored. Sometimes the effort of coming up with recipes with a frenetic lifestyle is too demanding and time consuming, which causes that a lot of food simply doesn't end up being used before it expires, creating food waste as well as being a waste of money.

This project's objective is to solve exactly this problem: The user points a phone camera at the fridge or kitchen counter, and the app detects the ingredients, estimates how much of each is present, and proposes recipes that fit. The interaction is one tap. The entire computer vision pipeline runs on-device so the app works without an internet connection for the detection and measurement steps, and only contacts a small cloud endpoint for recipe generation.

The technical interest lies in combining three computer vision capabilities that are rarely integrated together: real-time on-device object detection and classification, monocular metric depth estimation for absolute-scale measurement without any reference object, and a geometric pipeline that fuses both outputs into estimated ingredient weights. These three stages form the CV core of the project.

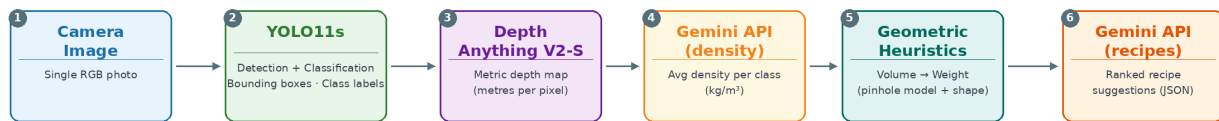
2. Project type

The problem described previously is related to productivity. It focuses on a deployable end-to-end pipeline (preprocessing → detection → depth → estimation → output), uses realistic data that reflects deployment conditions (real kitchen photos), has clear evaluation metrics for each stage, and ships as an installable mobile application. Performance, reliability and practical impact are the primary criteria.

3. Proposed CV techniques & methods

The CV pipeline has five stages. Stages ①②④⑤ run entirely on-device; stages ③ and ⑤ make lightweight cloud calls to Gemini 3.1 Flash Lite. The architecture is summarised in the table and diagram below.

Stage	Model / tool	Size	Where it runs	Output
① Detection + classification	YOLO11s (Ultralytics)	~20 MB	On-device (TFLite / CoreML)	Bounding boxes + class labels
② Metric depth estimation	Depth Anything V2-S (metric indoor)	~98 MB	On-device (ONNX Runtime Mobile)	Per-pixel depth map (metres)
③ Density lookup	Gemini 3.1 Flash Lite (API call)	—	Cloud (one call at app start)	Avg density per class in kg/m ³ (JSON)
④ Weight estimation	Pinhole model + shape heuristics	—	On-device (Dart)	Estimated weight per item (g)
⑤ Recipe generation	Gemini 3.1 Flash Lite (API call)	—	Cloud (one call per scan)	Ranked recipe suggestions (JSON)



On-device: ① Camera ② YOLO11s ③ Depth Anything V2-S ⑤ Geometric Heuristics Cloud (Gemini API): ④ Density lookup ⑥ Recipe generation

Figure 1. Full CV pipeline — on-device stages shown in colour; Gemini API cloud calls in amber/orange.

3.1 Detection and classification — YOLO11s

Object detection is performed by YOLO11s, the small variant of Ultralytics' YOLO11 family released in late 2024. YOLO11s gives a strong balance of accuracy and mobile-friendliness: 47.0 mAP50–95 on COCO at 2.46 ms GPU latency, around 9.4 million parameters, and an exported size of roughly 20 MB after INT8 quantisation. YOLO is anchor-free and natively outputs class labels along with bounding boxes, so a single model handles both detection and classification, which means that no second classifier head is needed.

An alternative considered was Roboflow's RF-DETR, particularly its new Nano and Small variants released in mid-2025. RF-DETR has higher COCO accuracy at comparable GPU latency thanks to its DINOv2 backbone and NMS-free decoder, but its file size (~120–130 MB) and immature mobile-export tooling make it a poor fit for an offline phone app at this stage. The detailed comparison is below.

Model	Params	mAP50–95 (COCO)	GPU latency	Size	Mobile export
YOLO11n	2.6 M	39.5	1.55 ms	~6 MB	Excellent (native)
YOLO11s ★ chosen	9.4 M	47.0	2.46 ms	~20 MB	Excellent (native)
YOLO11m	20.1 M	51.5	4.70 ms	~39 MB	Good (native)
RF-DETR-Nano	30.5 M	48.0	~2.3 ms	~120 MB	Limited (new)
RF-DETR-Small	32.1 M	53.0	~3.5 ms	~130 MB	Limited (new)

Decision: YOLO11s. It is the smallest model that retains strong accuracy, has a mature export path to TFLite and CoreML, and is well-supported by Flutter inference plugins.

3.2 Metric depth estimation — Depth Anything V2 Small

To estimate weight, the system needs to recover real-world size from pixels. The chosen approach is monocular metric depth estimation: a single neural network that, given an RGB image, outputs an absolute depth value in metres for every pixel without requiring a reference card or known camera intrinsics.

The model selected is Depth Anything V2 Small, in its metric-indoor variant (fine-tuned on Hypersim). It is roughly 98 MB, runs in ~30 ms on a desktop GPU, and is released under the Apache 2.0 licence — important because the Base and Large variants are CC-BY-NC and not deployable. The model is used pretrained; no fine-tuning is planned.

Model	Type	Size	GPU latency	Accuracy δ_1	License
MiDaS v3.1	Relative	~330 MB	~50 ms	0.68	MIT
ZoeDepth	Metric	~345 MB	~170 ms	0.75	MIT
Depth Anything V2-S ★ chosen	Metric*	~98 MB	~30 ms	0.82	Apache 2.0
Depth Anything V2-L	Metric*	~1.3 GB	~180 ms	0.89	CC-BY-NC
Depth Pro (Apple)	Metric	~2.5 GB	~300 ms	0.89	Apple sample code
Metric3D v2	Metric	~1.5 GB	~200 ms	0.91	MIT

	Relative depth	Metric depth ★ chosen
Output	Normalised map (0–1). Tells you which object is closer, not how far.	Absolute distance in metres per pixel.
Scale	Relative only — no real-world units.	Real-world units. No calibration needed.
Weight estimation	Impossible without a known reference object in the frame (coin, credit card).	Directly usable: $\text{real_width} = (\text{bbox_px} / \text{focal_px}) \times \text{depth_m}$.
User experience	Requires user to place a calibration object every scan.	One tap — no calibration object required.

3.3 Weight estimation — pinhole model, Gemini density, shape heuristics

Once YOLO11s provides bounding boxes and DA-V2-S provides a metric depth map, the system recovers real-world physical dimensions using the pinhole camera model:

$$\text{real_width} = (\text{bbox_width_px} / \text{focal_length_px}) \times \text{depth_m}$$

The focal length in pixels is read from the image's EXIF metadata when available, or approximated as $\text{image_width} \times 0.8$ as a default. The same formula applies to height and depth, yielding a real-world bounding volume per detected item.

Density via Gemini API

To convert volume to weight, the average bulk density (kg/m^3) of each detected class is needed. Maintaining a static hand-curated database would be incomplete and hard to maintain, instead, the system queries Gemini 3.1 Flash Lite whenever it encounters a class whose density is not yet cached locally. The prompt batches all new classes seen in a single scan:

"Return only a JSON object mapping each food class name to its average bulk density in kg/m^3 . Classes: tomato, apple, carrot, onion, milk_carton, pasta_box, olive_oil_bottle ... Include packaging weight for packaged goods. No explanation, only JSON."

The returned density values are cached locally in the app so that subsequent scans do not require an additional API call. This approach handles all class types uniformly — fresh produce (a tomato $\approx 950\text{--}1050 \text{ kg/m}^3$), bulk items, and packaged goods (where Gemini returns a density that accounts for the typical packaging + contents weight) — without requiring manual maintenance of any lookup table. When a class is genuinely ambiguous, Gemini returns a reasonable food-science estimate rather than failing.

Shape heuristics

A per-class shape heuristic maps the recovered bounding-box dimensions to a volume estimate: spheres for round produce (apple, orange, tomato), cylinders for elongated items (carrot, cucumber, bottle), and boxes for packaged goods (carton, can, jar). Multiplying the estimated volume by the Gemini-supplied density yields the weight in grams. The result is passed to the recipe prompt as a context hint, as Gemini is able to deal with approximate quantities, $\pm 30\%$ accuracy on weight is sufficient to generate appropriate recipe suggestions.

3.4 Recipe generation — Gemini 3.1 Flash Lite

After weight estimation, the app makes a second Gemini 3.1 Flash Lite call with the full list of detected ingredients and their estimated weights. The model returns three ranked recipe suggestions as a structured JSON array, which the app parses and displays directly.

LLM for recipe generation rather than a local SQLite database containing a recipe corpus is deliberate: the language model handles arbitrary ingredient combinations, adapts suggestions to available quantities ("only 120g of pasta, suggest a single-serving dish"), and returns natural-language cooking steps without requiring a curated recipe corpus. The density call is made lazily and cached, so in most sessions only the recipe call reaches the network.

4. Mobile application — Flutter (Android + iOS)

Flutter was chosen so a single Dart codebase produces both an Android APK and an iOS IPA, which is essential for a usable final product that does not depend on a specific platform. The mobile app is the delivery vehicle for the CV pipeline, not part of the pipeline itself.

Inference is integrated via two community packages: `tflite_flutter` runs the INT8-quantised YOLO11s TFLite model on both platforms, and `onnxruntime_flutter` runs the Depth Anything V2-S ONNX export. All preprocessing (resize, channel normalisation, tensor construction) is handled in Dart on a background isolate to keep the UI thread responsive. The two Gemini API calls are made via the official `google_generative_ai` Dart package.

The user flow is intentionally minimal: open the app, tap Scan, point at the fridge or kitchen counter, tap Capture. The pipeline runs, and within a few seconds the screen shows the annotated image with bounding boxes, the estimated ingredient list with weights, and the three recipe suggestions. No account, no server to manage, no subscription.

5. Expected outcomes & deliverables

The final delivery will include the following artefacts:

Deliverable	Description
Fine-tuned YOLO11s	Custom food-detection model trained on a curated multi-source dataset, exported to TFLite (Android) and CoreML (iOS)
Depth Anything V2-S (ONNX)	Pretrained metric-indoor checkpoint exported to ONNX for cross-platform inference via ONNX Runtime Mobile
Flutter mobile app	Cross-platform APK/IPA with camera capture, on-device inference, weight estimation, and recipe display
Evaluation report	Per-class mAP, weight-estimation error analysis, and qualitative depth comparison on real kitchen scenes
Slide deck + demo video	Presentation materials and a short screen-capture of the app running on a phone
Source code	Public GitHub repository with training scripts, export pipelines, Flutter app, and reproducible README

6. Methodology & timeline

The project follows the four phases mandated by the course guidelines. The internal milestones are sized so that Phase 2 (the 50 % checkpoint) shows a working but unpolished end-to-end pipeline running on a laptop, and Phase 3 delivers the mobile app and the full evaluation.

Phase	Deadline	Deliverable
Phase 1 — Definition	This week	Project definition: problem, stack, scope, evaluation plan
Phase 2 — Checkpoint 50%	26 or 28 May 2026	Trained YOLO11s on food dataset; depth + weight pipeline working on laptop; preliminary metrics
Phase 3 — Final delivery	15 or 17 June 2026	Final Flutter APK/IPA, full report, slides, demo video
Phase 3 — Presentation & Q&A	16 or 18 June 2026	Live demo on phone + presentation
Phase 4 — Peer evaluation	22 June 2026	Peer-to-peer grading

Per-phase work breakdown

- Phase 1 (now): finalise this proposal, set up the project repository, agree on the unified class list, and start dataset assembly on Roboflow.

- Phase 2 (by 26/28 May): fine-tune YOLO11s on Colab; export Depth Anything V2 to ONNX; implement the weight-estimation logic in Python; build a laptop-based Gradio prototype that demonstrates the full pipeline end-to-end.
- Phase 3 (by 15/17 June): port inference to Flutter using `tflite_flutter` and `onnxruntime_flutter`; integrate the Gemini API call; build the UI; run the evaluation; write the report and slides.
- Phase 3 presentation: live demo of the app running on a real phone, scanning real food.

7. Risks & mitigations

Risk 1 — ONNX Runtime Mobile may not support all ViT operators used by Depth Anything V2.

Mitigation: validate the ONNX export early (Phase 2 first week); if blocked, quantise the model to INT8 which often resolves operator compatibility, or fall back to running depth via a small bundled Python runtime on Android (Chaquopy) as a documented limitation.

Risk 2 — Weight estimation is fundamentally noisy from a single RGB image. Mitigation: this is explicitly framed in the report as a known limitation; the metric is reported with confidence intervals and the LLM recipe step is designed to tolerate rough quantities.

Risk 3 — Gemini free tier policy may change before delivery. Mitigation: the model string is a single configuration value; fallback options (Gemma 4 26B free tier at 1,500 RPD, or a local SQLite recipe database) are pre-identified and trivial to switch to.

8. Tools, novelty & AI usage

Tooling

- Training: PyTorch + Ultralytics on Google Colab (T4 GPU).
- Dataset: Roboflow Universe for sourcing, Roboflow workspace for merging and augmentation.
- Mobile inference: `tflite_flutter` (YOLO), `onnxruntime_flutter` (depth).
- LLM: Google Gemini 3.1 Flash Lite via the free tier of the Gemini API.
- AI coding assistants used throughout for boilerplate, as permitted by the course guidelines.

Novelty

The novelty of this project is not in any single model — every component is a well-known building block. It is in the integration: combining real-time on-device detection with on-device metric monocular depth to recover physical quantities, packaged into a one-tap mobile experience that anyone can relate to. To the knowledge of the team, no published student or open-source project combines these three CV stages (detection, metric depth, geometric weight recovery) on a mobile device with this user-facing flow.